

计组强化结课考试（试卷+答案）

注：本试卷难度、答题量略高于真题

一、某计算机中，`int` 为 32 位补码整数，`float` 采用 IEEE 754 单精度格式。执行：`float f = (float)x;`

1. `x` 的机器数为 `01000001H`。真值=16777217
2. `x` 的机器数为 `01000003H`。真值=16777219
3. `x` 的机器数为 `08000007H`。真值=134217735
4. `x` 的机器数为 `F7FFFFFFH`。真值=-134217737
5. `x` 的机器数为 `0FFFFFFFH`。真值=268435455

真 $\rightarrow$ float (舍入)

针对每个 `int` 变量，写出强制转换为 `float` 后的十六进制机器数。

▼ 参考答案

IEEE 754 单精度浮点数共有 24 位二进制有效数字，其中最高位 1 隐含，不写入尾数字段，机器数中的尾数字段实际只有 23 位。本题按 IEEE 默认的“就近舍入，舍入到偶数”处理。

进位条件为： $G = 1$ 且 ($R = 1$ 或 $S = 1$ 或 $L = 1$)

为便于探讨，约定：

..... L | G R S

- L：最后一个保留位。
- G：第一个舍去位（保护位）。
- R：第二个舍去位（舍入位）。
- S：R 后所有舍去位的按位或（粘滞位） $xxx | xxSxxxxx$
- 若不存在 R 或 S，则相应位按 0 处理。
或（只要有一个 1，S 即置 1）
- 下文黄色高亮的 24 bit 是舍入前准备保留的 24 位有效数字，其中包含最高有效位（尾数的隐含位）1。

1) `x = 01000001H = 00000001 00000000 00000000 00000001B`

符号位为 0，表示正数，因此补码、原码相同。去掉原码数值部分前面的无效 0，将保留 24bit 有效尾数：

$\overset{L}{1}0000000 \ 00000000 \ 00000000 \ | \ \overset{G}{1} \ \overset{R}{0} \ \overset{S}{0}$

黄色部分是准备保留的 24 位有效数字，舍去部分只有 1 位，因此：

$L = 0, G = 1, R = 0, S = 0$

进位条件 $G = 1$ 且 ($R = 1$ 或 $S = 1$ 或 $L = 1$) 不满足，因此 float 的“二进制科学计数法”可记为：

$1.000000000000000000000000 \times 2^{24}$

- 符号位：0
- 阶码真值： $e = 24$ ，移码阶码： $E = 24 + 127 = 151 = 10010111B$
- 尾数：`000000000000000000000000`

(1)
 $f = 0 \mid 10010111 \mid 00000000000000000000 = 4B800000H$ 。float 真值为 16777216。

2) $x = 01000003H = 00000001 \ 00000000 \ 00000000 \ 00000011B$

符号位为 0，表示正数，因此补码、原码相同。去掉原码数值部分前面的无效 0，将保留 24bit 有效尾数：

$10000000 \ 00000000 \ 00000001 \mid 1 \ 0 \ 0$
L G R S

黄色部分是准备保留的 24 位有效数字，舍去部分只有 1 位，因此：

$L = 1, G = 1, R = 0, S = 0$

进位条件 $G = 1$ 且 $(R = 1 \text{ 或 } S = 1 \text{ 或 } L = 1)$ 满足，因此保留的 24bit 有效尾数末位加 1：

$10000000 \ 00000000 \ 00000001 + 1 = 10000000 \ 00000000 \ 00000010$

因此 float 的“二进制科学计数法”可记为：

$1.0000000000000000000010 \times 2^{24}$

- 符号位：0
- 阶码真值： $e = 24$ ，移码阶码： $E = 24 + 127 = 151 = 10010111B$
- 尾数： 0000000000000000000010

(2)
 $f = 0 \mid 10010111 \mid 0000000000000000000010 = 4B800002H$ 。float 真值为 16777220。

3) $x = 08000007H = 00001000 \ 00000000 \ 00000000 \ 00000111B$
28位

符号位为 0，表示正数，因此补码、原码相同。去掉原码数值部分前面的无效 0，将保留 24bit 有效尾数：

$10000000 \ 00000000 \ 00000000 \mid 0111$
L G R S

黄色部分是准备保留的 24 位有效数字，舍去部分为 0111，因此：

$L = 0, G = 0, R = 1, S = 1$

进位条件 $G = 1$ 且 $(R = 1 \text{ 或 } S = 1 \text{ 或 } L = 1)$ 不满足，因此 float 的“二进制科学计数法”可记为：

$1.000000000000000000000000 \times 2^{27}$

- 符号位：0
- 阶码真值： $e = 27$ ，移码阶码： $E = 27 + 127 = 154 = 10011010B$
- 尾数： 000000000000000000000000

(3)
 $f = 0 \mid 10011010 \mid 00000000000000000000 = 4D000000H$ 。float 真值为 134217728。

4) $x = F7FFFFFFH = 11110111 \ 11111111 \ 11111111 \ 11110111B$

符号位为 1，表示负数。先将补码转为原码：

$[x]_{\text{原}} = 10001000 \ 00000000 \ 00000000 \ 00001001B$
28位

即 $x = -08000009H = -134217737$ 。去掉原码数值部分前面的无效 0，将保留 24bit 有效尾数：

$10000000 \ 00000000 \ 00000000 \mid 1001$
L G R S

黄色部分是准备保留的 24 位有效数字，舍去部分为 1001，因此：

$L = 0, G = 1, R = 0, S = 1$

进位条件 $G = 1$ 且 $(R = 1 \text{ 或 } S = 1 \text{ 或 } L = 1)$ 满足, 因此保留的 24bit 有效尾数末位加 1:

$10000000\ 00000000\ 00000000 + 1 = 10000000\ 00000000\ 00000001$

因此 float 的“二进制科学计数法”可记为:

$-1.000000000000000000000001 \times 2^{27}$

- 符号位: 1
- 阶码真值: $e = 27$, 移码阶码: $E = 27 + 127 = 154 = 10011010B$
- 尾数: 000000000000000000000001

$f = 1 \mid 10011010 \mid 000000000000000000000001 = CD000001H$ 。float 真值为 -134217744 。

5) $x = 0FFFFFFFH = 00001111\ 11111111\ 11111111\ 11111111B$

符号位为 0, 表示正数, 因此补码、原码相同。去掉原码数值部分前面的无效 0, 将保留 24bit 有效尾数:

$11111111\ 11111111\ 11111111 \mid 1111$

黄色部分是准备保留的 24 位有效数字, 舍去部分为 1111 , 因此:

$L = 1, G = 1, R = 1, S = 1$

进位条件 $G = 1$ 且 $(R = 1 \text{ 或 } S = 1 \text{ 或 } L = 1)$ 满足, 因此保留的 24bit 有效尾数末位加 1:

$11111111\ 11111111\ 11111111 + 1 = 1\ 00000000\ 00000000\ 00000000$

加 1 后产生进位, 需要重新规格化。因此 float 的“二进制科学计数法”可记为:

$1.000000000000000000000000 \times 2^{28}$

- 符号位: 0
- 阶码真值: $e = 28$, 移码阶码: $E = 28 + 127 = 155 = 10011011B$
- 尾数: 000000000000000000000000

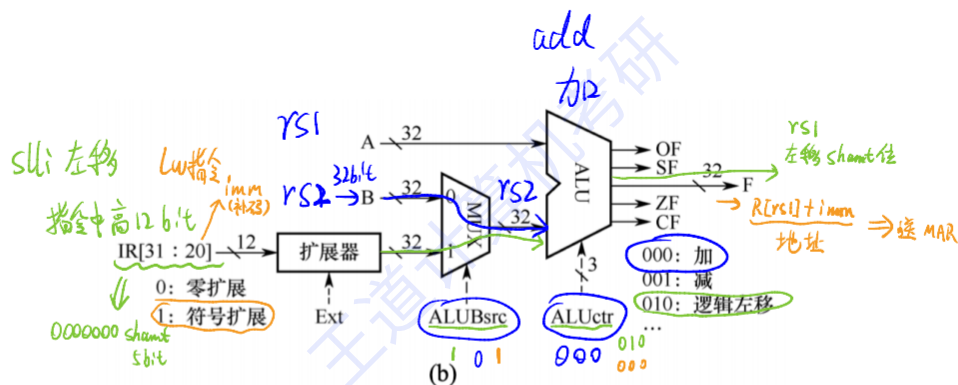
$f = 0 \mid 10011011 \mid 000000000000000000000000 = 4D800000H$ 。float 真值为 268435456 。

二、【2024 真题43改编】(13+4 分) 假定计算机 M 字长为 32 位, 按字节编址, 采用 32 位定长指令字, 指令 add、slli 和 lw 的格式、编码和功能说明如图 (a) 所示。

指令	31	25	24	20	19	15	14	12	11	7	6	0	指令功能说明
add	00000000			rs2		rs1		000		rd		0110011	$R[rd] \leftarrow R[rs1] + R[rs2]$
slli	00000000			shamt		rs1		010		rd		0010011	$R[rd] \leftarrow R[rs1] \ll \text{shamt}$
lw	imm					rs1		010		rd		0000011	$R[rd] \leftarrow M[R[rs1] + \text{imm}]$

(a)

其中, $R[x]$ 表示通用寄存器 x 的内容, $M[x]$ 表示地址为 x 的存储单元内容, shamt 为移位位数, imm 为补码表示的偏移量。图 (b) 给出了计算机 M 的部分数据通路及其控制信号 (用带箭头虚线表示), 其中, A 和 B 分别表示从通用寄存器 rs1 和 rs2 中读出的内容; $IR[31:20]$ 表示指令寄存器中的高 12 位; 控制信号 Ext 为 0、1 时扩展器分别实现零扩展、符号扩展, ALUctr 为 000、001、010 时 ALU 分别实现加、减、逻辑左移运算。请回答下列问题。



- 1) 计算机 M 最多有多少个通用寄存器？为什么 $shamt$ 字段占 5 位？（2 分）
- 2) 执行 add 指令时，控制信号 $ALUBsrc$ 的取值应是什么？若 $rs1$ 和 $rs2$ 寄存器内容分别是 $8765\ 4321H$ 和 $9876\ 5432H$ ，则 add 指令执行后，ALU 输出端 F 、 OF 和 CF 的结果分别是什么？若该 add 指令处理的是无符号整数，则应根据哪个标志判断是否溢出？（5 分）
- 3) 执行 $slli$ 指令时，控制信号 Ext 的取值可以是 0 也可以是 1，为什么？（2 分）
- 4) 执行 lw 指令时，控制信号 Ext 、 $ALUctr$ 的取值分别是什么？（2 分）
- 5) 若一条指令的机器码是 $A040\ A103H$ ，则该指令一定是 lw 指令，为什么？若执行该指令时， $R[01H] = FFFF\ A2D0H$ ，则所读取数据的存储地址是什么？（2 分）
- 6) 【改编新增】假定计算机 M 采用小端方式，地址 $FFFF\ 9CD4H \sim FFFF\ 9CD7H$ 中依次存放 $78H$ 、 $56H$ 、 $34H$ 、 $12H$ ，则执行上述（第 5 小问） lw 指令后，目的寄存器的内容是什么？（2 分）
- 7) 【改编新增】若执行 lw 指令时，控制信号 Ext 错误地取为 0，所形成的存储地址是否一定错误？请说明理由。（2 分）

▼ 参考答案

1) 从指令中格式中可见，寄存器编号用 5bit 表示，因此共有 $2^5=32$ 个通用寄存器。

$slli$ 指令实现逻辑左移功能，通用寄存器与机器字长同宽，即 32bit。 $shamt$ 字段表示左移位数，刚好用可 5bit 表示左移位数 $0 \sim 31$ 。（注：若左移位数 ≥ 32 ，则寄存器中的 32bit 将全部归零，移位将毫无意义）

2) add 指令实现两个寄存器 $rs1$ 、 $rs2$ 的加法运算，图 (b) 中，A、B 刚好对应寄存器 $rs1$ 、 $rs2$ ，因此 $ALUBsrc=0$

将两个 32 位数相加：

$$\begin{array}{r} 8765\ 4321H \\ + 9876\ 5432H \\ \hline 1\ FDB\ 9753H \end{array}$$

（最高位产生进位 $Cout=1$ ）

ALU 的输出端 F 只保留低 32 位，因此：

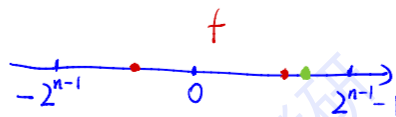
$$F = 1FDB\ 9753H$$

• CF（进位标志）的判断：

- 方法一（机器算）： $CF = \text{最高位产生的进位 } Cout \oplus sub = 1 \oplus 0 = 1$ 。（注： $sub=0$ 加法， $sub=1$ 减法）
- 方法二（人算）：将加数、被加数视为无符号数，相加后最高位出现了进位 1，即加法结果需要用 33 位才能表示，因此无符号数加法溢出， $CF = 1$ 。

• OF（溢出标志）的判断：

- 方法一（机器算）：次高位产生进位为 $C_{31} = 0$ ，最高位产生的进位 $Cout = C_{32} = 1$ ，因此



$$OF = C_{31} \oplus C_{32} = 0 \oplus 1 = 1$$

- 方法二（人算）：两个操作数的最高位均为 1，按补码解释时二者均为负数；结果 1FDB 9753H 的最高位为 0，表示正数。两个负数相加得到正数，故发生有符号溢出， $OF = 1$ 。

综上： $F = 1FDB\ 9753H$ ， $OF = 1$ ， $CF = 1$ ；无符号整数是否溢出应根据 **CF** 判断。

3) 因为 slli 指令的高 12 位 IR[31:20] 中，最高位固定为 0，因此零扩展、符号扩展效果等价。

4) lw 的执行效果是 $R[rd] \leftarrow M[R[rs1] + imm]$ ，其中 imm 是补码表示的偏移量，要进行符号扩展，因此 Ext=1；ALUctr=000。（具体原理：ALU 的 A 端输入寄存器 rs1 的 32bit，ALU 的另一端输入 imm 经过符号扩展后的 32bit 补码，控制信号 ALUBsrc=1，最终 ALU 执行加法运算，得到访存地址 $R[rs1] + imm$ ）

5) 将机器码 A040 A03H 按指令格式划分：

$$\underbrace{1010\ 0000\ 0100\ 00001}_{imm[11:0]} \quad \underbrace{010}_{rs1} \quad \underbrace{00010}_{funct3} \quad \underbrace{0000011}_{rd \quad opcode}$$

其中， $opcode=0000011$ ， $funct3=010$ ，与 lw 指令的编码完全一致，因此该指令一定是 lw 指令。各字段为：

- $imm = A04H$;
- $rs1 = 01H$;
- $rd = 02H$ 。

所以该指令的功能为：

$$R[02H] \leftarrow M[R[01H] + \text{SignExt}(A04H)]$$

立即数 A04H 是 12 位补码，其最高位为 1，表示负数。对其进行符号扩展可得：

$$\text{SExt}(A04H) = FFFF\ FA04H = -5FCH$$

因此，所读取数据的存储地址为：

$$\text{地址} = R[01H] + \text{SignExt}(imm) = FFFF\ A2D0H + FFFF\ FA04H = FFFF\ 9CD4H$$

故该指令所读取数据的存储地址是 FFFF 9CD4H。

6) lw 指令读取的是从地址 FFFF 9CD4H 开始的一个 32 位数据，即连续读取 4 个字节：

存储地址	FFFF 9CD4H	FFFF 9CD5H	FFFF 9CD6H	FFFF 9CD7H
存储内容	78H	56H	34H	12H
在 32 位数据中的位置	最低字节	次低字节	次高字节	最高字节

计算机 M 采用小端方式，低地址存放数据的低有效字节、高地址存放数据的高有效字节，因此组合得到的 32 位数据为：

$$\underbrace{12H}_{\text{最高字节}} \quad \underbrace{34H}_{\text{次高字节}} \quad \underbrace{56H}_{\text{次低字节}} \quad \underbrace{78H}_{\text{最低字节}} = 1234\ 5678H$$

由第 5 小问可知，该指令的目的寄存器为 $rd=02H$ ，因此执行指令后：

因此，执行指令后， $R[02H] = 1234\ 5678H$ 。

7) 不一定。是否形成错误地址，取决于 12 位立即数 imm 的最高位，即 $imm[11]$ ：

- 当 $imm[11]=0$ 时，立即数表示非负数。符号扩展和零扩展均在高 20 位补 0，二者结果相同，因此即使 Ext 错误地取 0，所形成的存储地址也不会出错。
- 当 $imm[11]=1$ 时，立即数表示负数。符号扩展应在高 20 位补 1，而零扩展会在高 20 位补 0，二者结果不同，因此会形成错误的存储地址。

例如，对于第 5 小问中的 lw 指令， $imm=A04H$ ，且 $imm[11]=1$ 。正确的符号扩展结果为：

$$SE_{ext}(A04H) = FFFF\ FA04H$$

若 Ext 错误地取 0，则零扩展结果为：

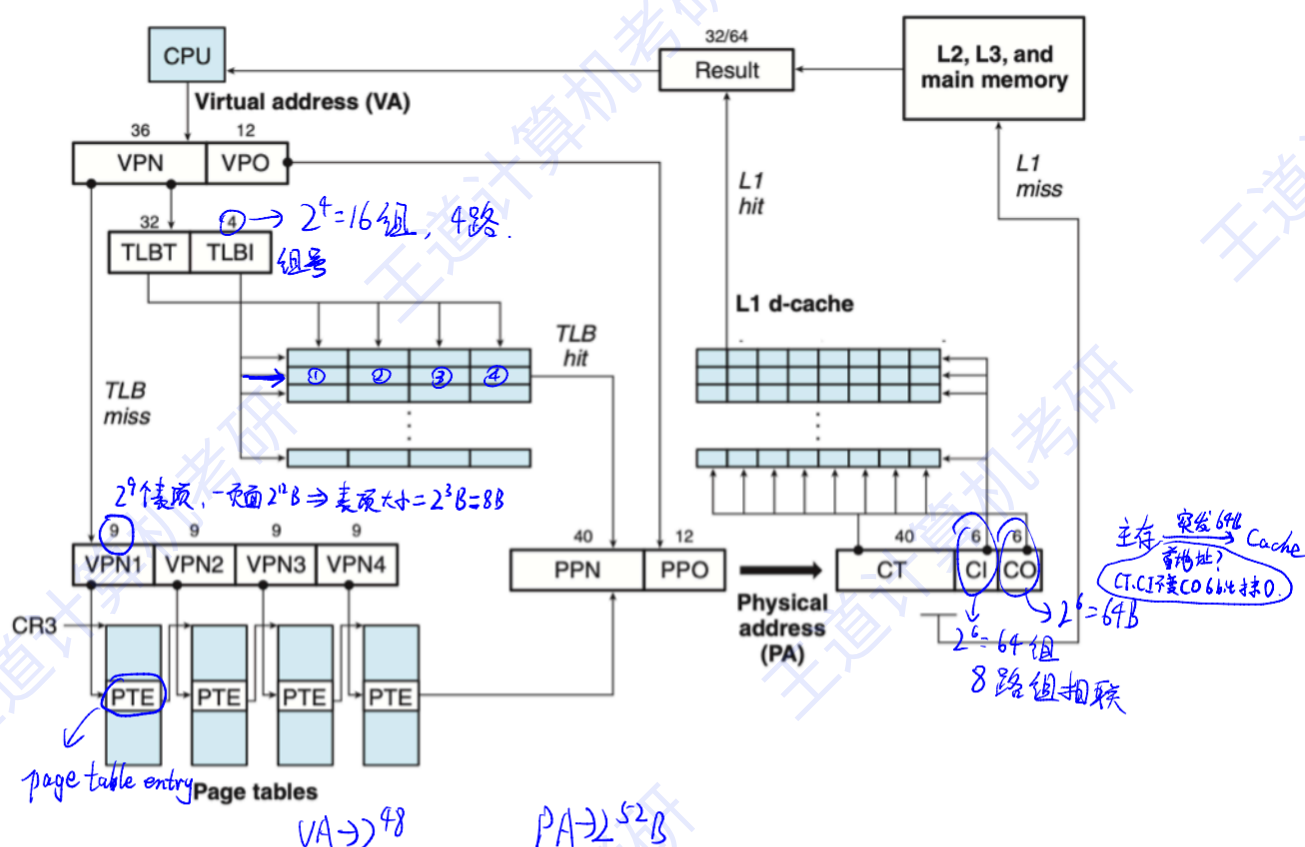
$$ZE_{ext}(A04H) = 0000\ 0A04H$$

此时形成的错误地址为：

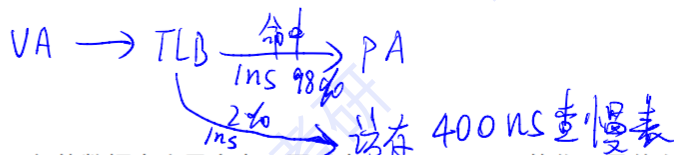
$$FFFF\ A2D0H + 0000\ 0A04H = FFFF\ ACD4H$$

而正确地址是 $FFFF\ 9CD4H$ 。所以对于第 5 小问中的这条具体指令，采用零扩展存储地址一定会出错。

三、下图是 Intel Core i7 地址转换过程的简要示意图，已知该机按字节编址，请根据图示回答下面的问题：



- 该CPU支持的虚拟地址空间大小是多少？物理地址空间大小是多少？
- 该处理器支持几级页表？每个页面的大小是多少？每个页表项的大小是多少？
- TLB采用什么映射方式？图示中 TLBT、TLBI 的作用是什么？请描述从虚拟地址转化为物理地址的过程。



- 4) Cache采用什么映射方式？每个Cache行的数据大小是多少？图示中CT、CI、CO的作用是什么？
- 5) 若Cache采用LRU替换策略，且采用写回法，请画出每个Cache行的结构，并计算图中L1 d-Cache的实际大小是多少？
- 6) 请描述根据物理地址访问图中Cache的过程。
- 7) 若TLB命中率为98%，TLB访问时间1ns，内存访问时间100ns。并假设当TLB miss时才开始访问内存中的页表，且各级页表常驻内存，则平均的地址转换时间是多少？
- 8) 【难，可不作】TLB访问时间1ns，内存访问时间100ns。并假设当TLB miss时才开始访问内存中的页表，只有顶级页表常驻内存。查多级页表的过程中，下一级页表未调入内存的概率是1%。已知处理一个缺页中断平均需要8ms。如果要满足平均地址转换时间小于2ns，那么TLB命中率需要至少多少？
- 9) 【难，可不作】假设访问某虚拟地址时，TLB未命中，已查到第四级页表，且第四级页表已经调入内存。访问目标物理地址的过程中，不考虑Cache，若内存访问时间100ns。当发生缺页时，若有一个可用的空页或被置换的页未被修改，则处理缺页并访问目标物理地址需要5ms；若被置换的页已被修改，则处理缺页并访问目标物理地址的时间需要10ms。请问为什么若被置换页面已被修改，处理缺页的时间开销会增加？若发生缺页时，60%的概率需要置换一个被修改的页面，为保证访问目标物理地址的平均时间不超过1ms，可接受的最大缺页中断率是多少？

▼ 参考答案

- 1) 看图可知，虚拟地址VA总位数为 $36+12=48$ bit，物理地址PA总位数为 $40+12=52$ bit。因此，虚拟地址空间大小为 $2^{48}B = 256TB$ ，物理地址空间大小为 $2^{52}B = 4PB$ 。

 注：408中通常约定K、M、G、T、P、E、Z相邻单位之间按 2^{10} 递增。因此， $2^{48}B = 256TB$ ， $2^{52}B = 4PB$ 。若严格采用IEC二进制单位，则应分别写作256 TiB和4 PiB。

- 2) 看图可知，该处理器支持4级页表。每个页面大小是 $2^{12}=4KB$ 。通常来说采用多级页表时，最理想的情况是每一级的页表大小不超过一个页面，而从图中可知，每一级的页号占9位，也就是说每一级页表有 2^9 个表项，因此每个页表项的大小为 $2^{12}/2^9 = 8B$ 。
- 3) TLB采用组相联映射，具体来说，4路组相联映射，共 $2^4 = 16$ 组。TLBT，即TLB Tag，TLB标记信息。TLBI，即TLB Index，索引号，也就是组相联映射中的“组号”，从图中可知，TLBI共4bit，说明总共有 $2^4 = 16$ 组。

地址转化过程如下：

查TLB

- 虚拟地址VA共48位，取出前36位作为虚页号VPN，VPN再拆分为32位TLBT、4位TLBI。
- 用4位TLBI确定TLB组号，一个分组中包含4个表项
- 用32位TLBT和分组中的4个表项对比，TLBT匹配，且有效位为1，说明TLB命中，直接得到物理页框号PPN（40bit）若TLB未命中，则查内存中的页表
 - 虚拟地址的前36位作为虚页号VPN。再拆分为 $9+9+9+9$ ，也就是4级页号
 - 根据每一级页号去查每一级页表。当然，查页表的过程有可能会缺页，如果缺页了还得调页
 - 查到最后一级页表，就可以确定物理页框号PPN（40bit）将物理页框号PPN和页内偏移量VPO拼接得到完整的物理地址PA

- 4) Cache采用组相联映射，具体来说，8路组相联映射，共 $2^6=64$ 组
每个Cache行的数据大小是 $2^6=64B$

不缺 $(1-P) \Rightarrow 100ns$

缺 $P \rightarrow \begin{cases} 40\% \text{ 有空页拒, 或不脏 } 5ms \\ 60\% \text{ 无空, 脏 } 10ms \end{cases}$

$(1-P) \times 100ns + P \times 0.4 \times 5ms + P \times 0.6 \times 10ms < 1ms$
 $\Rightarrow P_{max} \leq 12.5\%$

40位CT为Cache Tag，用于判断Cache是否命中

6位CI为Cache Index，用于表示Cache组号。其位数可以反映出总共有几个分组

6位CO为Cache Offset，也就是Cache字块内地址（块内偏移量）。其位数可以反映出每个Cache字块大小

5) 从图示可知，该Cache采用8路组相联映射，也就是说每个分组中有8个Cache行。若采用LRU替换策略，得用 3bit 来标记组内各个Cache行。采用写回法（write-back），需要 1bit 作为脏位。每个Cache行的结构如下：

Tag标记	有效位	LRU替换信息	脏位	数据
40bit	1bit	3bit	1bit	64B

图中的 L1 d-cache 共有 $8 \text{ 路} \times 64 \text{ 组} = 512 \text{ 行}$ 。每行的数据大小为 $64\text{B} = 512\text{bit}$ ，加上 40 bit Tag、1 bit 有效位、3 bit LRU 信息和 1 bit 脏位，每行实际大小为：

$$40 + 1 + 3 + 1 + 512 = 557 \text{ bit}$$

所以，L1 d-cache 的实际大小为：

$$512 \times 557 = 285184 \text{ bit} = 35648 \text{ B} = 34.8125 \text{ KB}$$

其中，纯数据区大小为 $512 \times 64\text{B} = 32\text{KB}$ ；其余 2.8125KB 为 Tag、有效位、LRU 信息和脏位等附加信息。

6) 物理地址 PA 为 $40+12 = 52\text{bit}$

- 就像图示那样，拆分为 40bit CT，6bit CI，6bit CO
- 根据 6bit CI 找到对应 Cache 分组，分组内共 8 个 Cache 行
- 物理地址的前 40bit CT 与分组内的 8 个 Cache 行的 Tag 标记对比，如果 Tag 能匹配，且有效位为 1，则 Cache 命中，找到一个 Cache 行
- 根据字块内偏移量，6bit CO，从 Cache 行的 64B 数据中，访问数据
- 如果 Cache 不命中，则需要根据物理地址 PA 去访存（当然，实际上在 i7 处理器中，还有 L2 级 Cache、L3 级 Cache，都没命中才会访问主存）

拓展：下面这个图是 core i7 页表项的结构，每个页表项 64bit，即 8B。看几个我们比较熟悉的东西：

63	62	52	51	12	11	9	8	7	6	5	4	3	2	1	0
XD	Unused	Page physical base addr				Unused	G	0	D	A	CD	WT	U/S	R/W	P=1

- P：存在位（Present），表示该页表项描述的页面或下一级页表是否已在内存中。P=0 时访问会触发缺页异常，P=1 时该页表项有效。
- R/W：读写权限位，用于表示该页面只读还是可读写。例如，存放常量的页面可设置为只读。
- U/S：用户/特权级权限位，用于表示用户态程序是否可以访问该页面。
- A：访问位（Accessed），表示对应页面或页表项是否被访问过，可供操作系统的页面置换算法参考。
- D：脏位（Dirty），表示页面内容是否被修改过。若被置换页面的 D=1，则需要先将其写回外存。

- 第 12~51 位共 40 bit：对于非叶子页表项，表示下一级页表的物理页框号；对于叶子页表项，表示目标物理页的物理页框号。

7) 若 TLB 命中，则可以直接完成地址转换，耗时 1ns。若 TLB 未命中，需要查询四级页表，每查一级页表需要一次访存，因此地址转换耗时 = 1+100+100+100+100=401ns。综上，平均地址转换时间 = $1 \times 98\% + 401 \times 2\% = 9 \text{ ns}$

8) 这个小问有点难，不用深究。通过这个小问，主要想提醒大家除了一级页表外，各级页表都有可能缺页：

- 若 TLB 命中，则可以直接完成地址转换，记耗时 $A=1\text{ns}$ 。设 TLB 命中的概率是 P 。
- 若 TLB 未命中（概率 $1-P$ ），需要查询四级页表，一级页表常驻内存。
 - 概率 $P_1 = (1-P) \times 0.01$ ——如果二级页表未调入内存，则三、四级页表也一定未调入内存，查页表过程中累计发生三次缺页。则耗时 $B = (1\text{ns}+100\text{ns}+8\text{ms})+(1\text{ns}+100\text{ns}+100\text{ns}+8\text{ms})+(1\text{ns}+100\text{ns}+100\text{ns}+100\text{ns}+8\text{ms})+(1\text{ns}+100\text{ns}+100\text{ns}+100\text{ns}+100\text{ns})$
 - 概率 $P_2 = (1-P) \times 0.99 \times 0.01$ ——如果二级页表原本已调入内存，第三级页表未调入内存，则四级页表也一定未调入内存，查页表过程中累计发生两次缺页。则耗时 $C = (1\text{ns}+100\text{ns}+100\text{ns}+8\text{ms})+(1\text{ns}+100\text{ns}+100\text{ns}+100\text{ns}+8\text{ms})+(1\text{ns}+100\text{ns}+100\text{ns}+100\text{ns}+100\text{ns})$
 - 概率 $P_3 = (1-P) \times 0.99 \times 0.99 \times 0.01$ ——如果三级页表原本已调入内存，则四级页表未调入内存的概率是 1%。查页表过程中累计发生一次缺页。则耗时 $D = (1\text{ns}+100\text{ns}+100\text{ns}+100\text{ns}+8\text{ms})+(1\text{ns}+100\text{ns}+100\text{ns}+100\text{ns}+100\text{ns})$
 - 概率 $P_4 = (1-P) \times 0.99 \times 0.99 \times 0.99$ ——如果四级页表原本已调入内存，则查页表过程中不会发生缺页，则耗时 $E = (1\text{ns}+100\text{ns}+100\text{ns}+100\text{ns}+100\text{ns})$

平均地址转换时间 = $P \times A + P_1 \times B + P_2 \times C + P_3 \times D + P_4 \times E < 2 \text{ ns}$ ，解出方程式即可得概率 TLB 命中率 P

9) 若被置换的页面被修改过，需要现将页面写回外存，再将缺失页面读入内存。“脏”页面写回外存的动作会增加开销。

查询到第四级页表，即可确定目标地址是否缺页。考虑两种情况：

- 概率 = $(1-P)$ ——不缺页。访问物理地址耗时 = 100 ns。
- 概率 = P ——缺页。此时需要细分为两种情况考虑
 - 概率 = $P \times 40\%$ ——有空闲页框，或被置换的页面“不脏”。处理缺页并访问目标物理地址耗时 5ms
 - 概率 = $P \times 60\%$ ——无空闲页框，且被置换的页面“脏”。处理缺页并访问目标物理地址耗时 10ms

综上，从已查询到四级页表开始算起，访问目标物理地址的平均耗时 = $(1-P) \times 100\text{ns} + P \times 0.4 \times 5\text{ms} + P \times 0.6 \times 10\text{ms} < 1\text{ms}$ 。解方程即可得答案

$$P_{\max} \approx 12.50\%$$

四、【2014 真题 44 改编】某程序中有如下循环代码段 P：“for(int i = 0; i < N; i++) sum += A[i];”。假设编译时变量 sum 和 i 分别分配在寄存器 R1 和 R2 中。常量 N 在寄存器 R6 中，数组 A 的首地址在寄存器 R3 中。程序段 P 起始地址为 0804 8100H，对应的汇编代码和机器代码如下表所示。

编号	地址 (VA)	机器代码 (4B)	汇编代码	注释
1	08048100H	00022080H	loop: sll R4, R2, 2	$(R2) \ll 2 \rightarrow R4$ (左移 $\times 2$)
2	08048104H	00832020H	add R4, R4, R3	$(R4) + (R3) \rightarrow R4$ (右移 $\div 2$)
3	08048108H	8C850000H	load R5, 0(R4)	$((R4) + 0) \rightarrow R5$ (地址)
4	0804810CH	00250820H	add R1, R1, R5	$(R1) + (R5) \rightarrow R1$ (Sum)
5	08048110H	20420001H	addi R2, R2, 1	$(R2) + 1 \rightarrow R2$ (Sum)
6	08048114H	1446FFFAH	bne R2, R6, loop	if $(R2) \neq (R6)$ goto loop

执行上述代码的计算机M采用32位定长指令字，其中分支指令bne采用如下格式：

31..26	25..21	20..16	15..0
OP	Rs	Rd	OFFSET

OP为操作码；Rs和Rd为寄存器编号；OFFSET为偏移量，用补码表示。请回答下列问题，并说明理由。

- M的存储器编址单位是什么？
字节 (16位 $\rightarrow$ 32位)
- 已知sll指令实现左移功能，数组A中每个元素占多少位？
4B, 32位
- 表中bne指令的OFFSET字段的值是多少？已知bne指令采用相对寻址方式，当前PC内容为bne指令地址，通过分析表中指令地址和bne指令内容，推断出bne指令的转移目标地址计算公式。
08048114H, PC + 4, OFFSET $\times$ 4
- 若M采用如下“按序发射、按序完成”的5级指令流水线：IF（取指）、ID（译码及取数）、EXE（执行）、MEM（访存）、WB（写回寄存器），且硬件不采取任何转发措施，分支指令的执行均引起3个时钟周期的阻塞，则P中哪些指令的执行会由于数据相关而发生流水线阻塞？哪条指令的执行会发生控制冒险？为什么指令1的执行不会因为与指令5的数据相关而发生阻塞？
写，后三条读同一个寄存器 $\rightarrow$ 分支 (改变PC)
- 【改编新增】模仿下图画出上述6条指令的指令流水线。

	时间单元													
指令	1	2	3	4	5	6	7	8	9	10	11	12	13	14
I ₁	IF	ID	EX	M	WB									
I ₂		IF	ID	EX	M	WB								
I ₃			IF				ID	EX	M	WB				
I ₄							IF				ID	EX	M	WB

▼ 参考答案

1)

因为每条指令长度为32位，占4个单元，所以存储器编址单位是字节。(2分)

【分析】一条指令占4B，观察表中各指令的地址可知，每条指令的地址差为4个地址单位，即4个地址单位代表4B，一个地址单位就代表1B，因此该计算机是按字节编址的。

2)

数组A中每个元素的地址通过下标左移两位（乘以4）再加数组首址得到，故每个数组元素占4B，即32位。(2分)

3)

OFFSET = FFFAH，值为-6。(1分) 指令bne所在地址为0804 8114H，转移目标地址为0804 8100H，因为0804 8100H = 0804 8114H + 4 + (-6) $\times$ 4，(1分) 所以，指令bne的转移目标地址计

算公式为：(PC) + 4 + OFFSET×4。(1分)

【分析】bne指令的机器代码为1446FFFAH，根据指令格式，后2B的内容为OFFSET字段。系统执行到bne指令时，PC自动加4（bne指令的长度）。

4)

由于数据相关而发生阻塞的指令为第2、3、4、6条，因此第2、3、4、6条指令都与各自的前一条指令发生数据相关。(3分) 第6条指令会发生控制冒险。(1分) 当前循环的第5条指令与下次循环的第1条指令虽有数据相关，但由于第6条指令后面有3个时钟周期的阻塞，因此消除了该数据相关。(1分)

5)



五、假定某输入设备A的速度为 n B/s，对应I/O接口中有一个 32位数据缓冲寄存器，请回答下列问题，并给出计算过程。

(1) 若设备A采用程序查询I/O方式，CPU每隔 t_1 会进行一次程序查询，每次程序查询时间开销为 t_2 （两次程序查询之间的时间间隔是 t_1 ，也就是说 t_2 是 t_1 的子区间），为了使设备A的输入数据不丢失，需要满足什么条件？



(2) 若设备A采用中断控制方式，中断处理的时间总开销为 t_3 ，则结合 (1) 中的条件，要让中断控制方式的效率比程序查询方式的效率更高，需要满足什么条件？

(3) 若设备A采用DMA控制方式，DMA预处理和后处理的时间总开销为 t_4 ，DMA传送块大小为 d 字节，假设DMA与CPU之间没有访存冲突，则要让DMA控制方式的效率比中断控制方式的效率更高，需要满足什么条件？

▼ 参考答案

(1) I/O接口的缓冲区大小 $32\text{bit} = 4\text{B}$ 数据。而设备的输入速度是 n B/s，因此每隔 $4/n$ 秒，缓冲区就会被充满，CPU要及时将数据取走，否则就可能导致数据丢失。因此需要满足 $t_1 \leq 4/n$

(2) 首先要理解什么叫“中断控制方式的效率更高”——就是采用中断控制方式的时候，CPU花在处理I/O上的时间比程序查询方式更少。我们不妨计算1秒之内，CPU花在处理I/O上的时间是多少。

若采用中断控制方式，则数据缓冲区每充满一次，就会产生一次中断，一秒之内产生中断的次数是 $n/4$ 次，每次中断处理时间开销为 t_3 。因此，一秒之内，CPU处理中断所花时间为 $(n/4)t_3$

若采用程序查询方式，一秒之内，CPU进行程序查询的次数为 $1/t_1$ ，每次程序查询耗时 t_2 。因此，一秒之内，CPU程序查询所花时间为 $(1/t_1)t_2$

综上，要使中断控制方式效率更高，需要满足 $(n/4)t_3 < (1/t_1)t_2$

(3) 要让 DMA控制方式 的效率更高，就是要保证采用 DMA方式时，CPU花在处理I/O上的时间更少。根据题目条件，我们不妨计算每传送 d 字节，CPU的时间开销。

若采用DMA控制方式，则每传送一块 d 字节，CPU用于DMA预处理和后处理的时间总开销为 t_4 。

若采用中断控制方式，则数据缓冲区每充满一次，就会产生一次中断，传送 d 字节，共产生中断 $d/4$ 次，每次中断处理时间开销为 t_3 。因此，每传送 d 字节，CPU用于中断处理的时间总开销为 $(d/4)t_3$

综上，要使DMA控制方式效率更高，需要满足 $t_4 < (d/4)t_3$

通过这个题目，可以感受到，三种I/O方式没有绝对的孰优孰劣，要综合考虑设备速度、CPU处理一次IO的开销。